

WEEK3 - COS20028 (BIG DATA ARCHITECTURE AND APPLICATION)

Dr Pei-Wei Tsai (Unit Convenor)
Yicun Tian (Lecturer, Tutor)

Two Updates

Data Science in Industry event

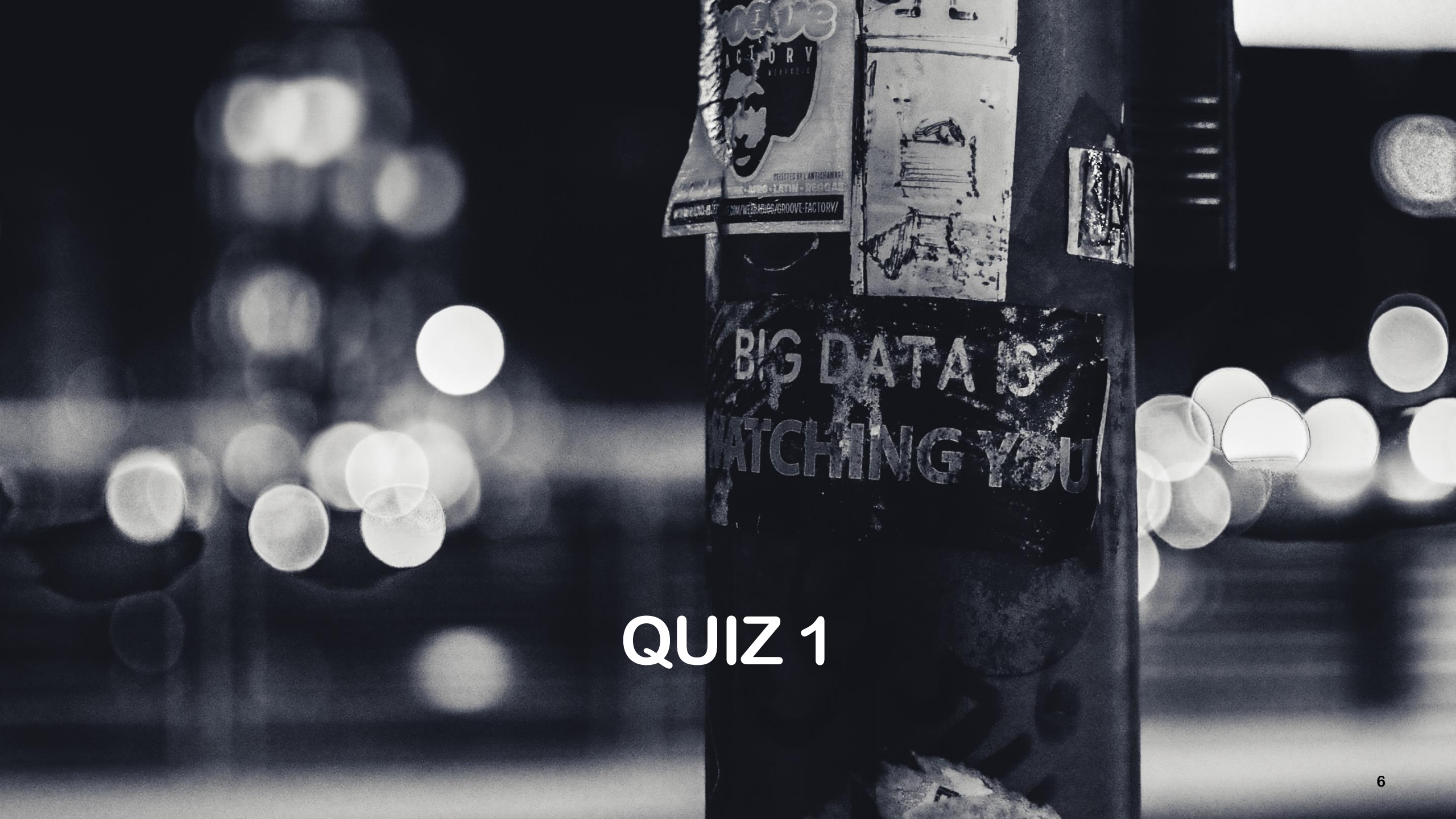
- Wednesday 20 August 5:00–6:30pm, TD121 and foyer area
- Open to current students only
- Join us for an evening featuring Swinburne alumni and current students sharing their experiences in the workplace. This event is a great opportunity to hear real-world insights, ask questions, and connect with peers.
- This is a kick-off event for the formation of a new student-led club for data science students, focused on running training sessions and hosting industry guests. Come along to show your support, help shape the vision and express your interest!
- RSVP via <https://forms.office.com/r/MUFe8HXZSc>; snacks and networking included!

Two Updates

- **Promote support services**
 - **Mental health support**
 - **Free GP access**
- **Wellbeing session option**
 - **Consider booking a batyr@Swinburne wellbeing session**
 - **Available any week this semester (details on the wiki)**

Week 3 Overview: What You'll Learn

Source	Content
Online Videos (Canvas)	Part 1: Concept of Writing a MapReduce Program in Java Part 2: Write a MapReduce Program in Java Part 3: Integrated Development Environments
Live Lecture	Quiz 1 Recap Java Concept Recap MapReduce Program Assignment 1
Lab Activities	Implementing a Word Count Program with Hadoop MapReduce



QUIZ 1



BIG DATA IS
WATCHING YOU

RECAP – JAVA CONCEPT

Java Recap

Category	Example	Key Concepts to Master
Class & Object	class, new IntWritable()	Class definition, object instantiation
Access Modifier	public, private	Java access control modifiers
Method Definition	public void reduce(...)	Return type, parameter types, scope
Type Conversion	value.get() returns an int	Accessing values from wrapper types
Java Annotation	@Override	Purpose of compiler annotations

Class & Object & Constructor – T/F

- 1) If you don't write any constructor in a Java class, the compiler will automatically provide a default no-argument constructor.
- 2) Once you define any constructor (e.g., a parameterized one), Java will still generate the default constructor for you.
- 3) You can create an object using `new ClassName()` only if the class has a no-argument constructor (either default or manually written)
- 4) A constructor must always have the same name as the class and cannot have a return type.
- 5) Constructors in Java can be overloaded, meaning you can define multiple constructors with different parameters.
- 6) The `new` keyword in Java is used to call a method, not to create an object.

Class & Object

- What is a Class?
- A class is a blueprint for creating objects.
- It defines fields (variables) and methods (functions).
- Classes enable encapsulation — bundling data and behavior.
- Class names use UpperCamelCase convention.
- Syntax

```
public class ClassName {  
    // fields and methods  
}
```

```
public class Person {  
    // Field (instance variable)  
    String name;  
  
    // Method to print the name  
    public void printName() {  
        System.out.println("Name: " + name);  
    }  
}
```

Class & Object

- What is an Object?
- An object is an instance of a class.
- It holds its own values for fields defined in the class.
- Use the new keyword to create an object
- Syntax

```
ClassName obj = new ClassName();
```

- Objects let us reuse code and store s
- Multiple objects can be created from

```
public class Person {  
    // Field (instance variable)  
    String name;  
  
    // Method to print the name  
    public void printName() {  
        System.out.println("Name: " + name);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Person p = new Person(); // Use the default constructor (automatically p  
        p.name = "Alice";         // Manually assign a value to the 'name' field  
        p.printName();            // Output: Name: Alice  
    }  
}
```

Class & Object - Constructor

- A constructor is a special method called when an object is created.
- It has the same name as the class and no return type.
- Used to initialize fields.
- Java provides a default constructor if none is written.
- Can create multiple constructors (constructor overloading).

Type	Has Parameters	Automatically Provided	Function
Default Constructor	No	Yes	Does nothing or assigns default values
Custom	Can have		Used to initialize fields

```
public class Person {  
    String name;  
  
    // Default constructor  
    public Person() {  
        // Optionally assign a default name  
        this.name = "Unnamed";  
    }  
  
    // Custom constructor  
    public Person(String name) {  
        this.name = name;  
    }  
}
```

```
Person p1 = new Person();           // Using the no-argument constructor  
Person p2 = new Person("Charlie");  // Using the parameterized constructor
```

Class & Object & Constructor – T/F

- 1) If you don't write any constructor in a Java class, the compiler will automatically provide a default no-argument constructor.  True
- 2) Once you define any constructor (e.g., a parameterized one), Java will still generate the default constructor for you.  False Explanation: The default constructor is only automatically provided if you define no constructor at all.
- 3) You can create an object using new ClassName() only if the class has a no-argument constructor (either default or manually written).  True
- 4) A constructor must always have the same name as the class and cannot have a return type.  True
- 5) Constructors in Java can be overloaded, meaning you can define multiple constructors with different parameters.  True
- 6) The new keyword in Java is used to call a method, not to create an object.  False Explanation: new is used to create an object (i.e., instantiate a class).

2. Java Access Modifiers– T/F

- 1) A field marked as public can be accessed from anywhere, including outside the class.
- 2) A field marked as private can still be accessed directly from another class in the same package.
Methods marked as public can be called from any other class.
- 3) Private fields can still be used inside the class where they are defined.
- 4) The printInfo() method can access the private field age in the Person class because it is inside the same class.

```
public class Person {  
    public String name;    // can be accessed from anywhere  
    private int age;       // can only be accessed within this class  
  
    public void printInfo() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age); // allowed: same class  
    }  
}
```

2. Java Access Modifiers

- Access modifiers control the
- Java provides four levels of access:
 - **public** → accessible from anywhere
 - **private** → accessible only within the class
 - (package-private) → default if no modifier is specified
 - **protected** → accessible within the package and subclasses
- Helps with encapsulation and data hiding
- Most commonly used: **public**

```
public class Person {  
    public String name;    // can be accessed from anywhere  
    private int age;       // can only be accessed within this class  
  
    public void printInfo() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age); // allowed: same class  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Person p = new Person();  
        p.name = "Alice";    // OK  
        // p.age = 25;       // ✗ ERROR: 'age' has private access  
        p.printInfo();       // OK: printInfo is public  
    }  
}
```

Java Access Modifiers– T/F

- 1) A field marked as public can be accessed from anywhere, including outside the class.  True
- 2) A field marked as private can still be accessed directly from another class in the same package.  False Private means strictly inside the same class only.
- 3) Methods marked as public can be called from any other class.  True
- 4) Private fields can still be used inside the class where they are defined.  True
- 5) The printInfo() method can access the private field age in the Person class because it is inside the same class.  True

```
public class Person {  
    public String name;    // can be accessed from anywhere  
    private int age;       // can only be accessed within this class  
  
    public void printInfo() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age); // allowed: same class  
    }  
}
```

3. Method Definition – T/F

- 1) Every method in Java must declare a return type, even if it returns nothing.
- 2) If no parameters are listed in the parentheses, the method cannot be called.
- 3) The access modifier of a method determines whether it can be used outside the class.
- 4) The method body is where the actual code runs when the method is called.
- 5) A method's name can be chosen freely, but it must follow Java naming rules (e.g., cannot start with a digit).

3. Method Definition

- A method defines a behavior or action of a class.

- Syntax:

```
accessModifier returnType methodName(parameters) {  
    // method body  
}
```

- Access Modifier (scope): controls visibility, e.g., public, private.
- Return Type: data type of the value the method returns (e.g., int, String, or void if no value is returned).
- Parameters: inputs the method requires, defined with type and name.
- Method Body: the actual code that runs when the method is called.

3. Method Definition

```
public class Calculator {  
  
    // Method with return type 'int' and two parameters  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    // Method with no return type (void) and one parameter  
    public void printResult(int result) {  
        System.out.println("Result: " + result);  
    }  
}
```

3. Method Definition – T/F

- 1) Every method in Java must declare a return type, even if it returns nothing. 
True (If it returns nothing, we use void.)
- 2) If no parameters are listed in the parentheses, the method cannot be called.  False (It can still be called, just without arguments, e.g., `printInfo()`.)
- 3) The access modifier of a method determines whether it can be used outside the class.  True
- 4) The method body is where the actual code runs when the method is called. 
True
- 5) A method's name can be chosen freely, but it must follow Java naming rules (e.g., cannot start with a digit).  True

4. Type Conversion

- 1) In Hadoop, `IntWritable` is used instead of `int` because it can be serialized across distributed systems.
- 2) The method `value.get()` in an `IntWritable` object returns a primitive `int`.
- 3) Wrapper classes like `IntWritable` and `Text` cannot be used in place of primitive types in Hadoop.
- 4) A primitive `int` can be directly stored in Hadoop's MapReduce framework without being wrapped.
- 5) Type conversion between wrapper types and primitives allows developers to work with familiar primitive operations while still using Hadoop's data structures.

4. Type Conversion

- In Java, primitive types (e.g., int, double) are basic data types.
- Hadoop and many frameworks use wrapper classes (e.g., IntWritable, DoubleWritable) instead of primitives.
- Wrapper classes make values serializable and usable in distributed systems.
- To access the underlying primitive value, we use methods like:
value.get() → returns the primitive int stored in an IntWritable.
- Type conversion is needed when moving between wrapper types and primitives.

```
IntWritable num = new IntWritable(5);  
int x = num.get();    // convert wrapper to primitive
```

4. Type Conversion

- 1) In Hadoop, IntWritable is used instead of int because it can be serialized across distributed systems.  True
- 2) The method value.get() in an IntWritable object returns a primitive int.  True
- 3) Wrapper classes like IntWritable and Text cannot be used in place of primitive types in Hadoop.  False(They are specifically designed to replace primitives in Hadoop.)
- 4) A primitive int can be directly stored in Hadoop's MapReduce framework without being wrapped.  False(It must be wrapped in IntWritable.)
- 5) Type conversion between wrapper types and primitives allows developers to work with familiar primitive operations while still using Hadoop's data structures.  True

5/6 Inheritance & Java Annotation

- 1) A subclass can access all private methods of its superclass.
- 2) The `extends` keyword is used in Java to create a class that inherits from another class.
- 3) If you override a method from a superclass, you must use the `@Override` annotation or the program will not compile.
- 4) The annotation is recommended but not required — the code still works without it.
- 5) The `@Override` annotation helps the compiler check if a method correctly overrides one from its superclass.
- 6) If a method name is misspelled and you write `@Override` above it, the compiler will show an error.
- 7) This is one of the main benefits of using `@Override`. A subclass can override a public method from its superclass to change its behavior.

5. Inheritance

- Inheritance allows a class to reuse fields and methods from another class.
- The extends keyword is used to create a subclass (child) of a parent (superclass).
- The subclass automatically has access to all the public and protected methods and fields of the superclass.
- Subclasses can also override methods of the superclass.
- Inheritance supports code reuse and is a fundamental concept in object-oriented programming.

```
class Animal {  
    public void makeSound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("Bark");  
    }  
}
```

```
class Dog extends Animal {  
    // new or overridden methods go here  
}
```

6. Java Annotation

- Annotations are special markers that give
- `@Override` tells the compiler that a method is overriding a method from a superclass.
- It helps catch mistakes — for example, missing a semicolon results in a compile-time error.
- It improves readability, showing clearly that a method is redefined.
- Commonly used in inheritance and polymorphism.

```
class Animal {  
    public void makeSound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("Bark");  
    }  
}
```

5/6 Inheritance & Java Annotation

- 1) A subclass can access all private methods of its superclass. ❌ False Private methods are not accessible in the subclass.
- 2) The extends keyword is used in Java to create a class that inherits from another class. ✅ True
- 3) If you override a method from a superclass, you must use the @Override annotation or the program will not compile. ❌ False
- 4) The annotation is recommended but not required — the code still works without it.
- 5) The @Override annotation helps the compiler check if a method correctly overrides one from its superclass. ✅ True
- 6) If a method name is misspelled and you write @Override above it, the compiler will show an error. ✅ True
- 7) This is one of the main benefits of using @Override. A subclass can override a public method from its superclass to change its behavior. ✅ True



BIG DATA IS
WATCHING YOU

RECAP DRIVER

Driver

```
package stubs;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

Driver

```
public class WordCount {  
    public static void main(String[] args) throws Exception {  
        if (args.length != 2) {  
            System.out.println("Usage: WordCount <input dir> <output dir>");  
            System.exit(-1);  
        } hadoop jar WordCount.jar stubs.WordCount input output  
    }  
}
```

```
Configuration conf = new Configuration();
```

```
Job job = Job.getInstance(conf, "Word Count");
```

```
job.setJarByClass(WordCount.class);
```

```
job.setMapperClass(WordMapper.class);
```

```
job.setReducerClass(SumReducer.class);
```

Driver

```
job.setJarByClass(WordCount.class);  
job.setMapperClass(WordMapper.class);  
job.setReducerClass(SumReducer.class);
```

```
→ { FileInputFormat.setInputPaths(job, new Path(args[0]));  
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
```

```
→ { job.setOutputKeyClass(Text.class);  
    job.setOutputValueClass(IntWritable.class);
```

```
→ { boolean success = job.waitForCompletion(true);  
    System.exit(success ? 0 : 1);  
    }
```

```
}
```

map 0% reduce 0%
map 50% reduce 10%
map 100% reduce 100%

T/F

- The driver class is responsible for defining the input/output paths and setting the Mapper and Reducer classes.
- `job.setJarByClass(WordCount.class);` tells Hadoop where to find the main class so it can distribute the job.
- If the number of command-line arguments is not exactly 2, the program still proceeds to set input/output paths.
- The line `System.exit(success ? 0 : 1);` ensures the program exits with a code that indicates success or failure.

T/F

- The driver class is responsible for defining the input/output paths and setting the Mapper and Reducer classes.

✓ True The driver configures the MapReduce job.

- `job.setJarByClass(WordCount.class);` tells Hadoop where to find the main class so it can distribute the job.

✓ True It helps Hadoop locate and launch the job correctly on the cluster.

- If the number of command-line arguments is not exactly 2, the program still proceeds to set input/output paths.

✗ False The program exits early using `System.exit(-1)` to avoid errors.

- The line `System.exit(success ? 0 : 1);` ensures the program exits with a code that indicates success or failure.

✓ True This is standard practice to signal program status to the OS or calling process.



BIG DATA IS
WATCHING YOU

RECAP MAPPER

Mapper

Its job is to read a line by line of words, and output a list of (word, 1).

Mapper handles:

- reading lines,
- splitting words,
- and emitting (word, 1) pairs.

```
public abstract class Mapper<KEYIN, VALUEIN, KEYOUT, VALUEOUT> {  
    protected void map(KEYIN key, VALUEIN value, Context context) {  
        // default implementation  
    }  
}
```

```
public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable> {  
    @Override  
    public void map(LongWritable key, Text value, Context context)  
        throws IOException, InterruptedException {  
        String line = value.toString();  
        for (String word : line.split("\\W+")) {  
            if (word.length() > 0) {  
                context.write(new Text(word), new IntWritable(1));  
            }  
        }  
    }  
}
```

"hello, world!"

line.split("\\W+")

["hello", "world"]

Reducer

```
String line = value.toString();
for (String word : line.split("\\W+")) {
    if (word.length() > 0) {
        context.write(new Text(word), new IntWritable(1));
    }
}
```

Input : "This is a test!"

Step 1: String line = value.toString(); → line = "This is a test!"

Step 2: line.split("\\W+") → ["This", "is", "a", "test"]

Step3: if(word.length > 0) {context.write(new Text(word), new IntWritable(1));}

("This", 1)

("is", 1)

("a", 1)

("test", 1)

T/F

- 1) The Mapper class in Hadoop requires four type parameters: input key type, input value type, output key type, and output value type.
- 2) In the WordMapper class, `Text word = new Text(token);` converts each word (String) into a Hadoop Text object before emitting it.
- 3) The `context.write(word, one);` method inside the mapper emits a single output key-value pair for each input line.

T/F

- The Mapper class in Hadoop requires four type parameters: input key type, input value type, output key type, and output value type.

✓ True These four types define how input data is read and how intermediate key-value pairs are emitted.

- In the WordMapper class, `Text word = new Text(token);` converts each word (String) into a Hadoop Text object before emitting it.

✓ True Hadoop's Text class is the wrapper for string data types.

- The `context.write(word, one);` method inside the mapper emits a single output key-value pair for each input line.

✗ False It emits one key-value pair for each token/word, not per line.



RECAP REDUCER

Reducer

- The Mapper outputs many (word, 1) pairs.
- The Reducer groups these by word, and gets a list of all the 1s for each word.
- This Reducer then adds up all the 1s and outputs (word, totalCount), where totalCount is how many times that word appeared.

```
WordMapper.java SumReducer.java ✖  
  
package stubs;  
  
import java.io.IOException;  
  
import org.apache.hadoop.io.IntWritable;  
import org.apache.hadoop.io.Text;  
import org.apache.hadoop.mapreduce.Reducer;  
  
public class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {  
    @Override  
    public void reduce(Text key, Iterable<IntWritable> values, Context context)  
        throws IOException, InterruptedException {  
        int wordCount = 0;  
        for (IntWritable value : values) {  
            wordCount += value.get();  
        }  
        context.write(key, new IntWritable(wordCount));  
    }  
}
```

("test", [1, 1, 1]) turns it into ("test", 3).

Reducer

Input
(a, 1)
(is, 1)
(is, 1)
(test, 1)
(test, 1)
(test, 1)
(This, 1)

```
int wordCount = 0;

for (IntWritable value : values) {
    wordCount += value.get();
}    wordCount = wordCount + value.get();

context.write(key, new IntWritable(wordCount));
```

Step 1: `int wordCount = 0` → `wordcount = 0`

Step 2: for loop →

First iteration: `value = 1` → `wordCount = 1`

Second iteration: `value = 1` → `wordCount = 2`

Third iteration: `value = 1` → `wordCount = 3`

Step3: `context.write(key, new IntWritable(wordCount));` → (test, 3)

Final output from `reduce()`

(a, 1) (is, 2) (test, 3) (This, 1)

T/F

- 1) In the reducer, `IntWritable result = new IntWritable(wordCount);` is used to create a new writable integer for the output.
- 2) The `Iterable<IntWritable> values` parameter in the reducer includes only one value per word.
- 3) The reducer's job is to simply add all the 1s associated with a particular word.

T/F

- In the reducer, `IntWritable result = new IntWritable(wordCount);` is used to create a new writable integer for the output.
 -  True Hadoop requires output values to be wrapped in its own data types like `IntWritable`.
- The `Iterable<IntWritable> values` parameter in the reducer includes only one value per word.
 -  False It includes all values (e.g., all the 1s) associated with the same word.
- The reducer's job is to simply add all the 1s associated with a particular word.
 -  True This is the essence of the word count example—counting occurrences of each word.



ASSIGNMENT 1